

File Handling in Python

25 January 2026 22:47

Introduction To Files

We have so far created programs in Python that accept the input, manipulate it and display the output. But that output is available only during the execution of the program and input is to be entered through the keyboard. This is because the variables used in the program that have a lifetime that last till the time the program is under execution. What if we want to store the data that were input as well as generated output permanently so that we can reuse it later? Usually organization would want to permanently store information about employees, inventory, sales, etcetera to avoid repetitive task of entering the same data. Hence data are stored permanently on secondary storage devices for reusability. We store Python program written in script mode with a .py extension. Each program is stored on a secondary device as a file. Likewise, the data entered and the output can be stored permanently into a file.

So what is a file? A file is a named location on a secondary storage media where data are permanently stored for later access.

Types of Files

Computers store every file as a collection of zeros and ones that is in binary form. Therefore, every file is basically just a series of bytes stored one after the other. There are mainly two types of data files - text file and binary file. A text file consists of human readable characters which can be opened by any text editor. On the other hand, binary files are made up of non-human readable characters and symbols which require specific programs to access its content.

Text file

A text file can be understood as a sequence of characters consisting of alphabets, numbers, and other special symbols. Files with extensions like .txt, .py, .csv, etc. are some examples of text files. File using a text editor (Example Notepad) we see several lines of text. However, the file contents are not stored in such a way internally. Rather, they are stored in a sequence of bytes consisting of zeros and ones. In ASCII or Unicode or any other encoding schemes, the value of each character of a text file is stored as bytes. For example, the ASCII equivalent value 65(1000001) will be displayed by a text editor as the letter 'A' since the number 65 in ASCII or in UNICODE character set represents 'A'.

Each line of a text file is terminated by a special character called end of line (EOL) For example, the default EOL character in Python is the new line('\n'). However, other characters can also be used to indicate EOL. When a text editor or a program interpreter encounters the ASCII equivalent of the EOL character, it displays the remaining file content starting from a new line. Contents in a text files are usually separated by white spaces, but, and tab are also commonly used to separate values in a text file.

Binary Files

Binary files are also stored in terms of bytes (zeros and ones) But unlike text files, these bytes do not represents the ASCII values of characters. Rather, they represent the actual content, such as image, audio, video, compressed version of other files, executable files, etc. These files are not human readable. Thus, trying to open a binary file using a text editor will show some garbage values. We need specific software to read or write the content of a binary file.

Binary files are stored in a computer in a sequence of bytes. Even a single bit change can corrupt the file and make it unreadable to the supporting application. Also it is difficult to remove any error which may occur in the binary file as a stored content are not human readable. We can read and write both text and binary files through Python programs.

Opening and Closing a Text File

In real world application computer programs deals with. Data coming from different sources like databases, CSV, files, HTML, XML, JSAON, etc. We broadly access files either to write or read data from it. But operation on files, including creating an, opening a file, writing data in a file, traversing a file, reading data from a file and so on. Python has the IO module that contains different functions for handling files.

Opening a file

To open a file in Python we use `open ()` function. The syntax of `open ()` is as follows:

```
file_object = open(file_name, access_mode)
```

This function returns a file object called file handle which is stored in a variable `file_object`. We can use this variable to transfer data to and from the Phi (Read and write) by calling the function defined in the pythons IO module. If the file does not exist, the above statement creates a new empty file and assigns it the name we specify in the statement.

The `file _object` has certain attributes that tells us basic information about the file, such as:

- `<file.closed>` Returns true if the file is closed and false otherwise.
- `<file.mode>` Returns the access mode in which the file was opened.
- `<file.name>` Returns the name of the file.

The `file _name` should be the name of the file that has to be opened. If the file is not in the current working directory, then we need to specify the complete path of the file along with its name.

The `axis _mode` is an optional argument that represents the mode in which the file has to be accessed by the program. It is also referred to as processing mode. Here mode means the operation for which the file has to be opened like. `<r>` For reading, `<w>` for writing, `<+>` for both reading and writing, `<a>` for appending at the end of an existing file. The default is the read mode. In addition, we can specify whether the file will be handled as binary (`<b>`) Or text mode. By default files are opened in text mode. That means strings can be read or written. Files containing non textual data are opened in binary mode. That means read or write are performed in terms of bytes. The file offset position in a table refers to a position of the file object when the file is opened on a particular mode.

Table: File open modes

File Mode	Description	File Offset Position
<code><r></code>	Opens the file in read only mode.	Beginning of the file.
<code><rb></code>	Opens the file in binary and read only mode.	Beginning of the file.
<code><r+></code> or <code><+r></code>	Opens the file in both Read and write mode.	Beginning of the file.

<code><w></code>	Open the file in write mode. If the file already exists, all the contents will be overwritten. If the file does not exist, then a new file will be created.	Beginning of the file.
<code><wb+></code> or <code><+wb></code>	Opens the file in read, write and binary mode. If the file already exist, the contents will be overwritten. If the file does not exist, then a new file will be created.	Beginning of the file.
<code><a></code>	Opens the file in append mode. If the file does not exist, then a new file will be created.	End of the file.
<code><a+></code> or <code><+a></code>	Opens the file in append and read mode. If the file doesn't exist then it will create a new file.	End of the file.

Consider the following example:

```
myObject = open("myfile.txt", "a+")
```

In the above statement the file myfile.Txt is opened in append mode and read modes. The file object will be at the end of the file. That means we can write data at the end of the file and at the same time we can also read data from the file using the file object name myObject.

Closing a file

Once we are done with the read write operations on a file, it is good practice to close the file. Python provides a close () method to do so. While closing a file, the system frees the memory allocated to it. The syntax of close() is:

```
file_object.close()
```

Here file _object is the object that was returned while opening the file.

Python makes sure that any unwritten or unsaved data is flushed off (Written) to the file before it is closed. Hence it is always advised to close the file once our work is done. Also, if the file object is reassigned to some other file, the previous file is automatically closed.

Opening a file using with clause

In Python, we can also open a file using the with clause.

The syntax of WITH clause is:

```
with open(file_name, access_mode) as file_object:
```

The advantage of using with clause is that any file that is opened using this clause is closed automatically, once the control comes outside the with clause. In case the user forgets to close the file explicitly or if an exception occurs, the file is closed automatically. Also, it provides the simpler syntax.

```
with open("myfile.txt", "r+") as myObject:
    content = myObject.read()
```

Here we do not have to close the file explicitly using the close () statement. Python will automatically close the file.

Writing to a Text File

For writing to a file, we first need to open it in write or append mode. Existing file in write mode. The previous data will be erased and the file object will be post recent at the beginning of the file. On the other hand, in append mode, new data will be added at the end of the previous data as the file object is at the end of the file. After opening the file, we can use the following methods to write data in the file.

- `write()` - for writing single string.
- `writeline()` - for writing a sequence of strings.

The `write()` method

`write()` Method takes a string as an argument and writes it to the text file. It returns the number of characters being written on a single execution of the `write()` method. Also we need to add a new line character (`'\n'`) at the end of every sentence to mark the end of line.

```
myobject = open("myfile.txt", "w")
myobject.write("Hello! I have started using files in Python.\n")
myobject.close()
```